

MDM Light — End-to-End Architecture

A CAP + Fiori Elements recreation of S/4HANA's Manage Business Partner app (F3163), rebuilt as a governed request-and-approval facade: nothing a requester types reaches S/4 until an approver signs off on it.

Runtime SAP BTP, Cloud Foundry

Backend SAP CAP (Node.js)

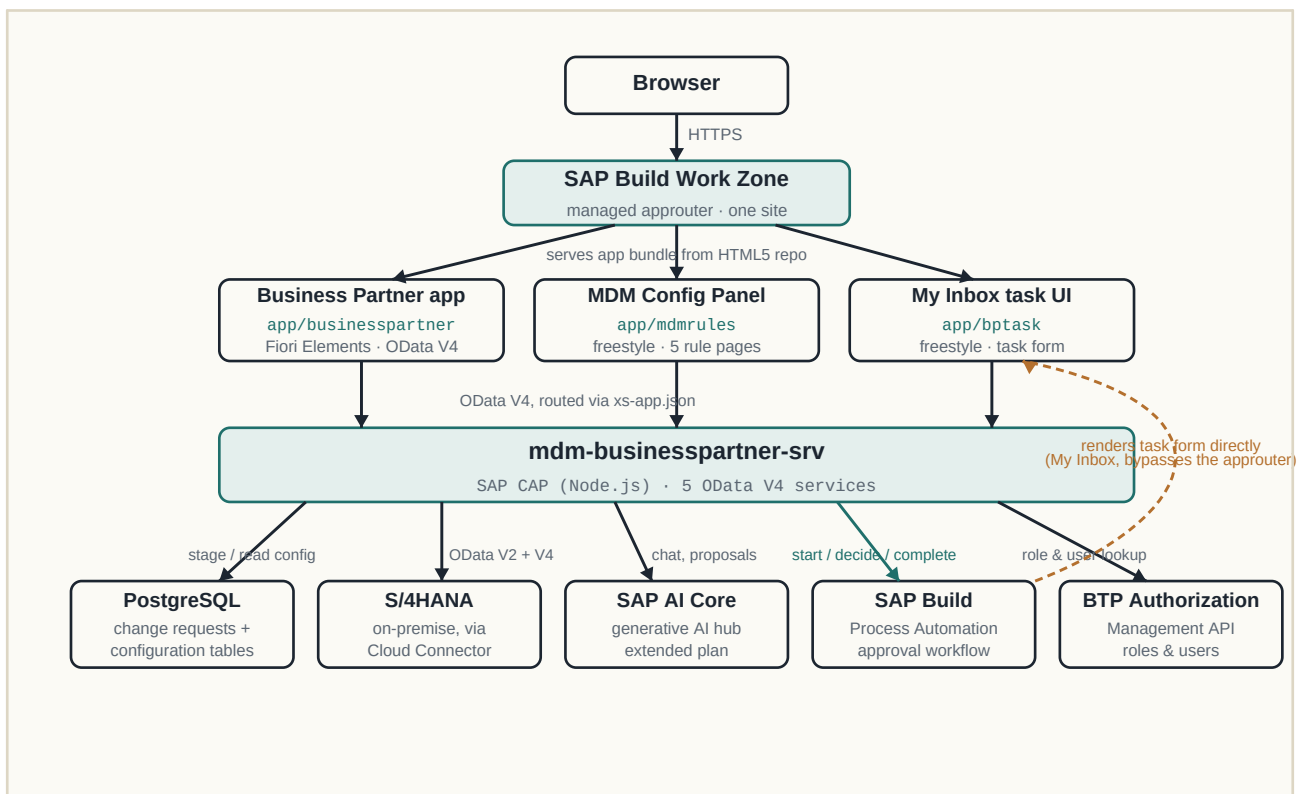
Frontend SAPUI5 / Fiori Elements OData V4

System of record S/4HANA on-premise

01 — THE WHOLE PICTURE

System landscape

One CAP service sits at the centre, fronting three SAPUI5 applications on one side and five external systems on the other. Every UI request travels through the managed approuter that SAP Build Work Zone provides.



Three SAPUI5 apps, one CAP backend, five connected systems. The task UI is the one exception to “everything goes through Work Zone”: SAP Build Process Automation's My Inbox resolves and renders it directly by `sap.cloud.service + sap.app.id`.

Applications

Four UI codebases, three deployed apps. The actual maintenance screen lives in a shared library and is copied — not installed as a dependency — into the two apps that render it.

Business Partner app `app/businesspartner`
Fiori Elements List Report / Object Page, OData V4, extended with custom controllers for the assistant, list actions, and deep links.

Search, request creation, and the Change Requests list.

MDM Configuration Panel `app/mdmrules`

Freestyle SAPUI5, five sub-pages: Duplicate Check, Validation, Derivation, Field Property, and Workflow Agent Determination rules.

Its own app rather than a second tile, since Work Zone standard edition exposes only one inbound per app.

My Inbox task UI `app/bptask`
Freestyle SAPUI5, built to SAP's UI5 Task Form contract. Renders approve, reject, rework and data-steward review inside My Inbox.

Declares no inbound at all — SBPA resolves it by service + app id.

Maintenance screen (shared library) `app/reuse`

One freestyle controller and view: create, edit, approve, rework, data-steward review, read-only display — one screen, six modes.

Copied at build time into both consuming apps; never resolved at runtime.

Backend services

One SAP CAP application (`mdm-businesspartner-srv`, Node.js) exposes five OData V4 services. It holds no Business Partner data of its own — reads are a live pass-through to S/4.

SERVICE	PATH	WHAT IT DOES
BusinessPartnerService	<code>/service/businesspartner</code>	Live facade over S/4's BP master data; the merged search list; the AI assistant; the full-screen maintenance actions.
ChangeRequestService	<code>/service/changerequest</code>	Staged create/change requests, the validate→derive→duplicate-check pipeline, save/submit/withdraw, approval and rework.
DuplicateConfigService	<code>/service/duplicateconfig</code>	Five configuration tables: duplicate, validation, derivation, field-property and workflow-agent rules.
CviConfigService	<code>/service/cviconfig</code>	Read-only Customer/Vendor Integration customizing — account groups, role categories, number ranges.
DerivationConfigService	<code>/service/derivationconfig</code>	Read-only SPRO customizing (language, tax categories, time zones, partner functions) the derivation stage proposes from.

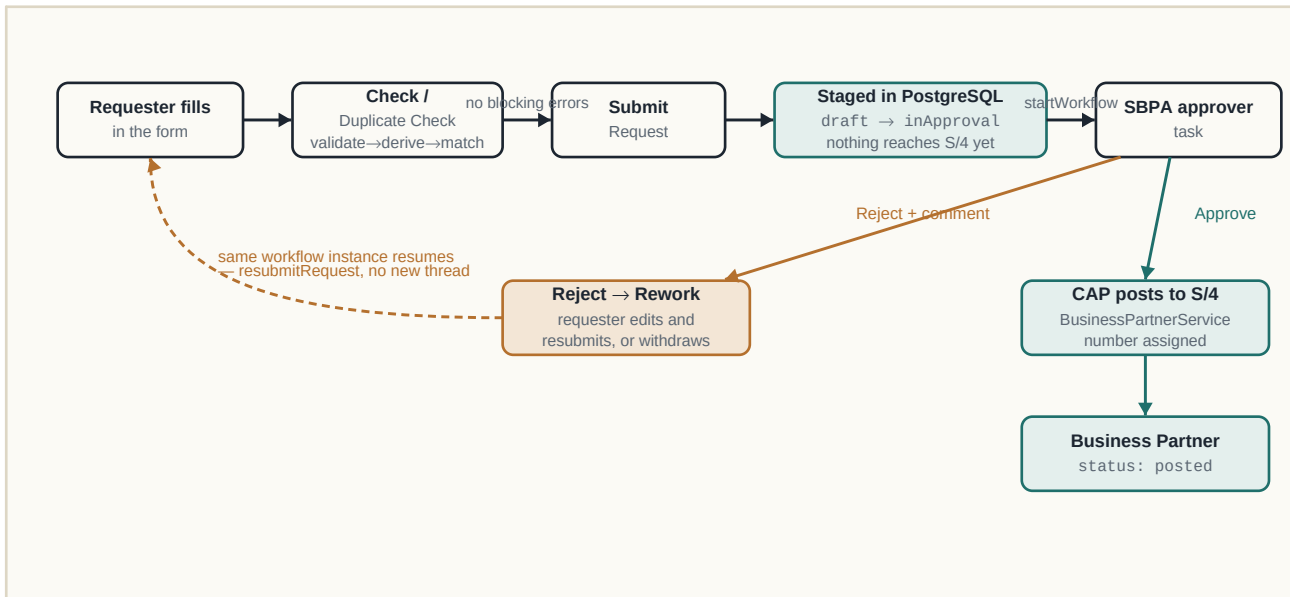
External systems & integrations

Every one of these is best-effort by design: an unreachable registry, a down AI service, or a stalled workflow call degrades the feature it backs, but never blocks the requester from saving their work.

SYSTEM	CONNECTION	PURPOSE
S/4HANA — Business Partner API	OData V2, VF_S4HANA_DEST, on-premise via Cloud Connector	Master data read/write (API_BUSINESS_PARTNER) and every F4 value help (ZSRVB_MDMLIGHT_VH).
S/4HANA — standard checks	OData V4, VF_S4HANA_V4_DEST	Runs SAP's own ZMDML_BPCHECK BP validations against the accepted request before approval.
PostgreSQL	CAP database binding	The only data this app owns: staged requests + five configuration tables.
SAP AI Core	@sap-ai-sdk/orchestration, extended plan	Assistant chat; duplicate/derivation proposals. Primary model claude-4.5-haiku, with fallbacks.
SAP Build Process Automation	REST API, OAuth2 client-credentials	The approval workflow engine. One process instance per request, parked across every rework round.
BTP Authorization Management API	second, apiaccess-plan XSUAA instance	Role collections and users for the approver/steward pickers and role-based field visibility.
VIES / GLEIF	public HTTP APIs	VAT validation and company-registry lookups, feeding the duplicate check and the assistant.
Wikipedia / DuckDuckGo	public HTTP APIs	Company research for the assistant's prefill — outranked by a confirmed registry hit.

The request lifecycle

The single decision the rest of this application is built around: a Business Partner is staged, checked, and routed for approval — and only posted to S/4 once somebody has signed off on exactly what will be created.



A rejection is a loop, not an end: the parked SBPA instance is resumed on resubmit rather than a fresh workflow being started, so one company's request keeps one audit thread however many rework rounds it takes.

Key design decisions

The choices worth defending in a design review — each one traded something away on purpose.

1 A live facade, not a database

There is no local Business Partner table; every read is a live projection over S/4's own API_BUSINESS_PARTNER OData V2 service. **Trade-off:** S/4 stays the single source of truth with zero replication lag — at the cost of every screen load depending on S/4 being reachable.

2 Stage, check, approve — then post

A create or change request is written to PostgreSQL first; only once an approver accepts it does CAP post to S/4. **Why:** before this existed, the workflow started only after the partner already existed in S/4 — the approver was reviewing something already live.

3 One merged search list

The list report reads a computed entity merging live S/4 partners with requests still in flight, so nobody requests the same company twice. **Trade-off:** the computed status columns are filtered/sorted in memory; sorting on them stays disallowed everywhere.

4 The maintenance screen is copied, not deployed

The requester's and approver's screens are the exact same controller, copied at build time into each app's webapp/reuse folder. **Why not a UI5 library:** a version-stamped library URL going stale is what 404'd the task UI in practice.

5 Managed approuter, not standalone

The app is served entirely through Work Zone's managed approuter; the standalone module was removed. **Why:** the BAS workflow-UI generator requires the managed approuter's markers and cannot coexist with a standalone one.

6 Three apps, one sap.cloud.service

The rules tile and the task UI are each their own HTML5 app, sharing one destination, app-host, and XSUAA scope. **Why:** Work Zone standard edition exposes only the first crossNavigation inbound per app — a second is silently dropped.

7 The approval task form is freestyle, not Fiori Elements

The My Inbox screen depends on no Fiori Elements library at all. **Why:** SAP documents UI5 task forms for freestyle apps only — hosting Fiori Elements inside a BPA task frame isn't a supported combination.

8 Validate → derive → duplicate-check, in that order

Every check runs the same three stages, always in this sequence. **Why:** data that fails validation cannot meaningfully be compared for duplicates, and derivation must fill a field before a rule can condition on it.

9 Behaviour lives in configuration tables, not code

Five decision tables — duplicate, validation, derivation, field visibility, workflow routing — are edited by a data steward, not a developer. **Trade-off:** an unreadable table must report itself rather than silently defaulting to “everything passes”.

10 The AI assistant is grounded, not general-purpose

Local, deterministic matching runs first; AI Core is a fallback within a bounded field allowlist that never includes bank or tax data. **Why non-reasoning:** a reasoning model can spend its whole budget on hidden reasoning and return nothing.

11 A confirmed registry hit outranks an AI-derived guess

VIES and GLEIF feed both the duplicate check and the assistant's suggestions; a confirmed VAT number always wins. **Why GLEIF is last resort:** a name search alone once matched a Belgian company to a Dutch entity's number.

12 Field visibility resolves against the caller's own BTP role

What a screen shows/hides/requires is the join of every matching profile, resolved against the specific BTP role collection the approver actually carries. **Why:** two broad “Approver” profiles couldn't tell two different approvers apart.

Deployment topology

One multi-target application, five modules, nine bound resources. Two build steps do work mbt cannot be trusted with here: a custom script zips the three UI apps by hand, and the CAP service builds from gen/srv, never srv/ directly.

MTA MODULE	TYPE	ROLE
mdm-businesspartner-srv	nodejs	The CAP backend — all five OData services, one process.
mdm-businesspartner-db-deployer	nodejs, one-off task	Evolves the PostgreSQL schema; refuses to run if it would drop a column.
mdm-businesspartner-app-content	com.sap.application.content	Ships all three UI bundles into one HTML5 app-host, zipped by a custom script.
mdm-businesspartner-destination-content	com.sap.application.content	Registers the app-host and the CAP service as destinations Work Zone can resolve.
BOUND RESOURCE	SERVICE / PLAN	WHY IT'S THERE
mdm-businesspartner-db	postgresql-db, free	Staging + configuration tables — the only persisted state this app owns.
mdm-businesspartner-auth	xsuaa, application	Authenticates users into the app itself.
mdm-businesspartner-authgmt	xsuaa, apiaccess	A second XSUAA instance — the only one with access to the BTP Authorization Management API.
mdm-businesspartner-repo-host / -runtime	html5-apps-repo	Stores and serves the three app bundles the managed approuter fronts.
mdm-businesspartner-destination-service	destination, lite	Resolves VF_S4HANA_DEST and the app-host/service destinations.
mdm-businesspartner-connectivity	connectivity, lite	The tunnel to the on-premise S/4 system via Cloud Connector.
mdm-businesspartner-aicore	aicore, extended	SAP AI Core access for the assistant and normalisation proposals.
mdmlight-bpa-key / -uaa	user-provided	API key and OAuth2 credentials for calling SAP Build Process Automation.

MDM Light — architecture reference compiled from the project's CAP model, MTA descriptor, and CLAUDE.md design log.